

HOW TO CHOOSE YOUR NEW HOME AT VANCOUVER AS A UBC STUDENT (OVERCOMPLICATED)

Introduction

I created this project as way to apply what I learned during my Master of Business analytics study. I grew interest in linear programming subjects – taught by Professor Schechter, and how it can optimally help us decide on the most mundane things in our life. In this case, it would be how we best choose our place as a UBC student. Combining with data visualisation class and web scrapping programming method, I created this dashboard as a way for me to apply what I have learnt in class to solve real life problem.

Project Overview

Helping UBC students find the best balance between rent, commute, and safety

Finding affordable and safe housing near UBC is a persistent challenge for students. Our project builds a **data-driven web application** that scores every postal code in Greater Vancouver on key factors that matter most — **rent affordability, commute time, and neighborhood safety**.

The tool helps students answer the question:

“Given my budget, which area offers the best overall living value for me?”

Category	Source	Example Fields
Rent Data	Craigslist	Average rent by room type, neighbourhood
Safety Data	City of Vancouver Open Data	Crime rate per 1,000 residents
Transit Data	TransLink GTFS	Commute time from postal code centroid to UBC campus

Spatial Context	Vancouver zoning, census population	Density, land use, demographics
------------------------	--	---------------------------------

Methods

- Multi-criteria scoring model**
Normalize and weight each factor (rent, commute, safety) → compute composite “Liveability Score.”
- Normalization** via *min–max scaling* or *winsorization* to reduce outliers
- Visualization:** mapping each neighbourhood price and crime rate
- Optimization:** Linear programming to maximize safety/affordability under a commute constraint.

Interactive Web Dashboard + API

- Enter budget → get **Top 5 postal codes** ranked by score
- Enter max modes and tradeoff appetite
- Explore results on **user input** with filters (budget, commute, safety priority)
- “How this score works” info page for transparency
- Optimal Decision Results

Week	Focus Area	Key Deliverables
W1 — Data & Design Foundations	- Finalize scoring metrics and weighting framework (safety, affordability, commute). - Set up repository, database schema, and environment (Python + Gurobi + PostGIS/SQLite). - Ingest and clean base datasets: City of Vancouver crime data, Craigslist, TransLink GTFS. - Define spatial unit (postal code or neighborhood).	Integrated raw dataset + project data dictionary.

W2 — Normalization & Market Insight Integration	• Apply winsorization and min–max normalization for each quantitative metric. • Design and distribute a student housing survey (Google Forms or Qualtrics) capturing: – Housing priorities (price vs. safety vs. commute) – Preferred neighborhoods – Tolerance levels for rent and commute. • Incorporate survey aggregates as weight priors or a “student sentiment” layer influencing (w_s, w_a, w_c). • Generate preliminary visualization (heatmap of preferences).	Clean, normalized dataset + survey summary + first version of weighted scoring model.
W3 — Optimization & API Layer	• Implement Gurobi binary optimization model: – Hard safety constraint ($s_i \geq s_{\min}$) – Soft budget & commute penalties (only for excess values) – Top-K recommendations. • Build backend API (Flask/FastAPI) allowing real-time solver runs based on user inputs. • Write unit tests for score computation + solver logic.	Functional optimization backend returning Top-K shortlist.
W4 — Dashboard & User Testing	• Develop interactive web dashboard (map + sliders for budget, commute, and weights). • Integrate backend API for live optimization. • Conduct user acceptance testing with students, analyze feedback, and fine-tune λ-penalties and scoring weights. • Deploy MVP (e.g., Streamlit or web app) + documentation + refresh scripts.	Deployed MVP + testing report + update pipeline guide.

Linear Programming Formulation

User-input variables (things you set at run time):

- K : number of neighbourhoods to return (Top-K picks).
- w_t : willingness-to-trade for commute time, in **CAD per minute** (you set $w_t = 50$).
- $C_{\max}$: maximum allowed crime rate, in **incidents per 1,000 residents** (you set $C_{\max} = 50$); this is a **hard constraint**.
- δ_m : penalty per extra transit mode (soft penalty in the objective). Larger δ_m discourages 2–3 mode routes more strongly.

Sets and indices

Neighbourhoods $i \in \{1, 2, \dots, N\}$.

Parameters (raw metrics only)

For each neighbourhood i :

- p_i = median rent (CAD)
- t_i = commute time to UBC (minutes)
- c_i = crime rate (incidents per 1,000 residents)
- m_i = number of modes

User / policy parameters:

- K = number of neighbourhoods to select
- $w_t = 50$ (CAD per minute willingness-to-trade)
- $C_{\max} = 50$ (hard crime cap, incidents per 1,000)
- $\delta_m \geq 0$ (soft penalty per extra mode)

Big-M:

- $M_C = \max \{0, \max_i (c_i - C_{\max})\}$ (any sufficiently large value also works)
-

Decision variables

For each neighbourhood i :

- $x_i \in \{0, 1\}$ = 1 if neighbourhood i is selected, else 0
-

Objective (minimize total “cost” in CAD-equivalent units)

$$\min \sum_{i=1}^N (p_i + w_t t_i + \delta_m (m_i - 1)) x_i$$

With the willingness-to-trade:

$$\min \sum_{i=1}^N (p_i + 50 t_i + \delta_m (m_i - 1))x_i$$

Constraints

1) Select exactly K

$$\sum_{i=1}^N x_i = K$$

2) Hard crime constraint (enforced only if selected)

$$c_i \leq C_{\max} + M_C(1 - x_i)\forall i$$

With your cap:

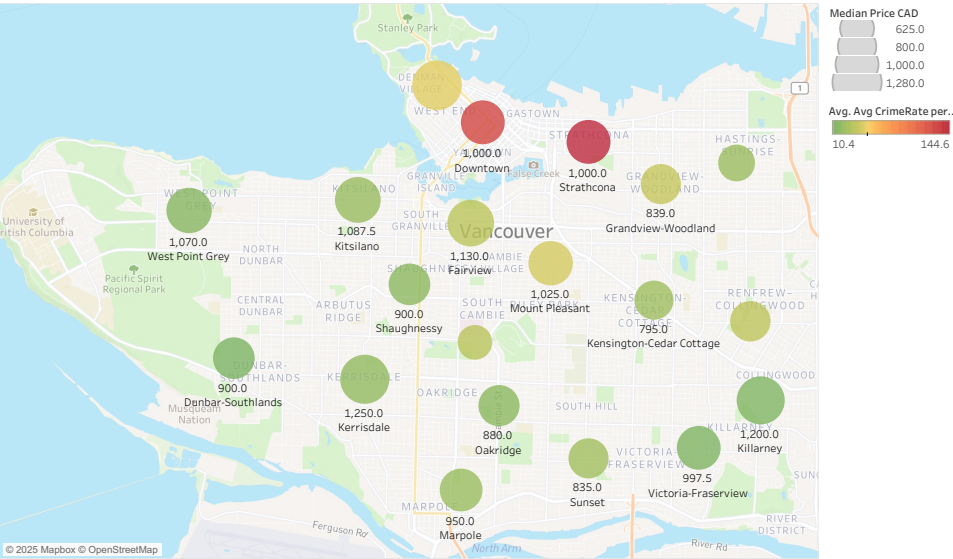
$$c_i \leq 50 + M_C(1 - x_i)\forall i$$

3) Integrality

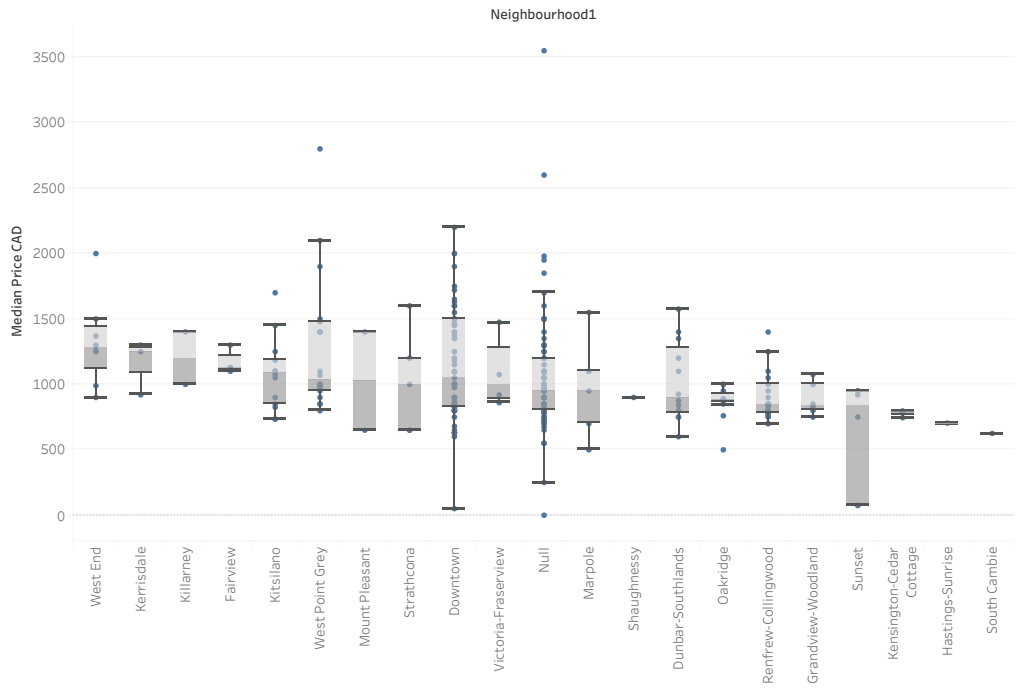
$$x_i \in \{0,1\}\forall i$$

Results

Mapping



Spread



Median of Price CAD for each Neighbourhood1. Details are shown for Property1.

Neighbourhood1	Avg. # Modes	Avg. CrimeRate per 1000 2025 ..	Duration to UBC (mins)	Avg. Price CAD
Null				1,042
Downtown	2	128	45	1,117
Dunbar-Southlands	1	10	14	1,014
Fairview	1	35	33	1,177
Grandview-Woodland	1	38	68	884
Hastings-Sunrise	1	22	84	704
Kensington-Cedar Cottage	1	23	44	778
Kerrisdale	1	19	33	1,157
Killarney	1	12	52	1,200
Kitsilano	1	21	25	1,075
Marpole	2	22	44	960
Mount Pleasant	1	43	52	1,025
Oakridge	1	17	34	862
Renfrew-Collingwood	1	35	59	909
Shaughnessy	1	16	25	900
South Cambie	1	34	35	625
Strathcona	2	145	67	1,020
Sunset	2	25	34	620
Victoria-Fraserview	2	12	53	1,083
West End	3	46	56	1,321
West Point Grey	1	14	15	1,313

The crime hard constraint (must be **< 50 incidents per 1,000**) knocks out only **Downtown (~128)** and **Strathcona (~145)**. Every other neighbourhood in the table is “eligible” on safety. Within the eligible set, the safest-looking areas are **Dunbar-Southlands (~10)**, **Killarney (~12)**, **Victoria-Fraserview (~12)**, **West Point Grey (~14)**, **Shaughnessy (~16)**, and **Oakridge (~17)**, while **West End (~46)** is close to the cap and should be treated as borderline.

Commute and price show the real trade-off: commute ranges from **~14–15 mins** (Dunbar-Southlands, West Point Grey) up to **84 mins** (Hastings-Sunrise), and average price ranges from about **\$620–\$625** (Sunset, South Cambie) up to **\$1,321–\$1,313** (West End, West Point Grey). Most neighbourhoods have **1 mode**, but **West End (3 modes)** and several **2-mode** areas (e.g., Sunset, Marpole, Victoria-Fraserview) will be penalized under your modes preference; combined with your high value on time, long-commute areas like **Hastings-Sunrise (84 mins)** will usually lose even if they look cheap.

Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] – Darwin 24.5.0 24F74)

CPU model: Apple M4

Thread count: 10 physical cores, 10 logical processors, using up to 10 threads

Optimize a model with 21 rows, 20 columns and 40 nonzeros

Model fingerprint: 0xa539db21

Variable types: 0 continuous, 20 integer (20 binary)

Coefficient statistics:

Matrix range [1e+00, 9e+01]

Objective range [2e+03, 5e+03]

Bounds range [1e+00, 1e+00]

RHS range [5e+00, 1e+02]

Found heuristic solution: objective 12550.000000

Presolve removed 21 rows and 20 columns

Presolve time: 0.00s

Presolve: All rows and columns removed

Explored 0 nodes (0 simplex iterations) in 0.00 seconds (0.00 work units)

Thread count was 1 (of 10 available processors)

Solution count 2: 10282.5 12550

Optimal solution found (tolerance 1.00e-04)

Best objective 1.028250000000e+04, best bound 1.028250000000e+04, gap 0.0000%

Neighbourhood	Median_Price_CAD	Duration	CrimeRate	Modes	ObjCost
Dunbar-Southlands	900.0	14	10.352464	1	1600.0
West Point Grey	1070.0	15	14.243415	1	1820.0
Shaughnessy	900.0	25	16.410187	1	2150.0
Kitsilano	1087.5	25	21.144118	1	2337.5
South Cambie	625.0	35	33.796200	1	2375.0

The optimizer's **Top-5 under the crime cap (<50/1,000)** are: **Dunbar-Southlands**, **West Point Grey**, **Shaughnessy**, **Kitsilano**, and **South Cambie**. All five are **1-mode** routes (so no modes penalty kicking in), and all have crime well below the threshold (roughly **10–34 per 1,000**). Commute times are mostly tight (**14–25 mins**) except **South Cambie (35 mins)**, which still makes the cut because it's much cheaper.

The pattern is clear: the model is prioritizing **short commutes and safety**, then using price to break ties. **Dunbar-Southlands** wins because it's very safe and the fastest commute (14 mins) at a reasonable price, while **West Point Grey** stays in despite being expensive because its commute is also extremely short (15 mins) and crime is low. **South Cambie** is the "value" pick—its commute is longer and crime is higher than the others in the list, but the **low rent (~\$625)** compensates enough to keep it in the Top-5.


```

import numpy as np

from gurobipy import Model, GRB

# df must have columns: Median_Price_CAD, Duration, CrimeRate, Modes
p = df["Median_Price_CAD"].to_numpy(dtype=float)
t = df["Duration"].to_numpy(dtype=float)
c = df["CrimeRate"].to_numpy(dtype=float)
modes = df["Modes"].to_numpy(dtype=float)

N = len(df)

K = 5

Cmax = 50.0

w_time = 50.0 # $ per minute willingness-to-trade

# Soft modes penalty (not a constraint)
# Calibrate: 1 extra mode ~ 15% of typical commute disutility
t_ref = float(np.median(t))
delta_m = 0.15 * w_time * t_ref # you can tune up/down

MC = max(0.0, float(np.max(c - Cmax))) # big-M for crime constraint

mdl = Model("raw_metrics_topK_soft_modes")
x = mdl.addVars(N, vtype=GRB.BINARY, name="x")

# Objective: rent + $/min * commute + modes penalty

```